C# CONSOLE APPLICATION ASSESSMENT QUESTIONS

Five Business Use Cases

50 Students • 5 Use Cases • 10 Students per Use Case

Common requirements for all five applications

- Build a menu-driven C# .NET Console Application.
- Implement every operation listed under the assigned use case.
- Use classes, properties, constructors, access modifiers, namespaces, and appropriate OOP design.
- Demonstrate inheritance and polymorphism, and use an interface or abstract class where appropriate.
- Use suitable collections such as List<T>, Dictionary<TKey,TValue>, HashSet<T>, Queue<T>, Stack<T>, or IEnumerable<T>.
- Implement and use at least one generic class or generic method.
- Use a delegate or Action<T>, Func<T,TResult>, or Predicate<T>, together with a suitable lambda expression.
- Use LINQ for filtering, projection, sorting, grouping, and aggregation as required by the use case.
- Validate user input and business rules. Implement exception handling and at least one suitable custom exception.
- Save and load the application's main data using JSON serialization and deserialization with System.Text.Json.
- Organize code into suitable model and service classes rather than putting all logic in Program.cs.
- Do not implement extension methods, asynchronous programming (async/await), or event handling.

USE CASE 1 — Hospital Appointment Management System

Assigned students: Students 1–10

Question

A clinic needs a console application to maintain patient records, doctor profiles, and appointments. Staff must book and manage appointments while preventing conflicting schedules and producing operational reports.

Required data entities

Patient: PatientId, FullName, Age, Gender, PhoneNumber

Doctor: DoctorId, FullName, Specialization, ConsultationFee, IsAvailable

Appointment: AppointmentId, PatientId, DoctorId, AppointmentDateTime, Status, FeeAtBooking

Required operations

A. Patient management

- Register a patient with a unique PatientId and all required details.
- Display all patients and display one patient by ID.
- Search patients by ID and by partial or full name.
- Update patient name, age, gender, and phone number.
- Delete a patient only when no active appointment would be left referencing that patient.

B. Doctor management

- Add a doctor with unique DoctorId, name, specialization, consultation fee, and availability.
- Display all doctors and retrieve a doctor by ID.
- Search doctors by name and specialization.
- Update doctor profile, specialization, consultation fee, and availability.
- Delete a doctor only when no scheduled appointment would be left without a doctor.

C. Appointment management

- Book an appointment for an existing patient with an existing available doctor.
- Generate a unique AppointmentId and capture the consultation fee at booking time.
- Display all appointments and retrieve an appointment by ID.
- Search appointments by patient ID, doctor ID, date, and status.
- Reschedule a scheduled appointment to a new valid date/time.
- Cancel a scheduled appointment and update its status.
- Mark a scheduled appointment as Completed.
- Prevent overlapping scheduled appointments for the same doctor.

D. Data persistence

- Save patients, doctors, and appointments to JSON files.
- Load saved data when the application starts and preserve references through IDs.
- Handle missing files as a first-run condition and handle malformed JSON without silently replacing saved data.

Business rules and validation

- Patient ID, Doctor ID, and Appointment ID must be unique.
- Patient age must be between 0 and 120; consultation fee must be greater than zero.
- Appointment date/time must be in the future when booking or rescheduling.
- A doctor marked unavailable cannot receive a new appointment.
- Only Scheduled appointments can be rescheduled or cancelled; only Scheduled appointments can be marked Completed.
- Cancelled appointments must not block a future booking slot.
- Invalid menu input must not crash the application.

Required reports using LINQ

- Find all appointments for a given doctor on a selected date.
- Find appointments by status and sort them by appointment date/time.
- Group appointments by doctor and count each doctor's scheduled appointments.
- Project a report containing appointment ID, patient name, doctor name, date/time, and status.
- Calculate revenue from Completed appointments using the fee stored at booking.
- Find the doctor with the greatest number of completed appointments.

USE CASE 2 — Restaurant Management System

Assigned students: Students 11–20

Question

A restaurant needs a console application to maintain menu items, customers, orders, table assignments, billing, and sales reports. Orders may contain multiple items and must follow valid preparation and service states.

Required data entities

MenuItem: ItemId, Name, Category, Price, IsAvailable, IsVegetarian

Customer: CustomerId, FullName, PhoneNumber

RestaurantTable: TableId, Capacity, IsOccupied

Order: OrderId, CustomerId (optional for walk-in), TableId (optional for takeaway), CreatedAt, Status, Items

OrderItem: ItemId, Quantity, UnitPriceAtOrder

Required operations

A. Menu management

- Add a menu item with unique ID, name, category, price, availability, and vegetarian flag.
- Display all menu items and retrieve an item by ID.
- Search items by name or category.
- Update item name, category, price, and availability.
- Delete a menu item without corrupting historical order details.
- Mark menu items as unavailable without deleting them.

B. Customer management

- Register a customer with unique CustomerId, name, and phone number.
- Display all customers and search by ID or name.
- Update customer details.
- Delete a customer when no business rule prevents it.

C. Table management

- Add and display restaurant tables with unique TableId and capacity.
- Assign a table to a dine-in order only when the table is not occupied.
- Release a table when the associated order is completed or cancelled.
- Reject a party size that exceeds table capacity if party size is captured.

D. Order and billing management

- Create an order as dine-in or takeaway.
- Add multiple available menu items and quantities to an order.
- View all orders, view one order by ID, and search by status, date, table, or customer.
- Increase or decrease an item's quantity and remove an item before finalizing the order.
- Calculate subtotal, discount, tax, and final amount using a clearly documented calculation order.
- Update order status through Placed, Preparing, Ready, and Served; support cancellation under valid rules.
- Prevent unavailable items and non-positive quantities from being added.
- Preserve each item's unit price at the time of order so later menu price changes do not alter old bills.

E. Data persistence

- Save menu items, customers, tables, orders, and order items to JSON.
- Load saved data at startup and retain order line items and historical prices.
- Handle missing files and malformed JSON safely.

Business rules and validation

- All IDs must be unique; prices and quantities must be positive.
- Only available items may be added to a new or editable order.
- An order must contain at least one item before it can be finalized.
- Define and enforce when an order becomes immutable, such as when preparation starts.
- Only valid order-status transitions are allowed; a Served or Cancelled order cannot return to preparation.
- Discount and tax rates must be non-negative and stated in the application.
- Historical bills must not change when menu prices are updated.
- Two active dine-in orders cannot occupy the same table.

Required reports using LINQ

- List menu items below a chosen price and sort by price.
- Project an order summary with order ID, status, subtotal, discount, tax, and final amount.
- Calculate total sales from Served orders only.
- Group orders by status and count orders in each group.
- Find the best-selling menu item by total quantity ordered.
- Find the highest-value order and calculate average served-order value.

USE CASE 3 — Gym Membership Management System

Assigned students: Students 21–30

Question

A gym needs to manage member profiles, membership plans, subscriptions, and payment records. Staff must renew memberships, identify expired subscriptions, and report revenue and membership trends.

Required data entities

Member: MemberId, FullName, Age, PhoneNumber, JoinDate

MembershipPlan: PlanId, PlanName, DurationInDays, Fee, IsActive

Membership: MembershipId, MemberId, PlanId, StartDate, EndDate, Status

Payment: PaymentId, MembershipId, Amount, PaymentDate, PaymentMethod, Status

Required operations

A. Member management

- Register a member with unique MemberId and required contact details.
- Display all members and retrieve a member by ID.
- Search members by name or phone number.
- Update member name, age, and phone number.
- Delete a member only if membership and payment history remains consistent; otherwise reject deletion.

B. Membership plan management

- Add a membership plan with unique PlanId, name, duration, fee, and active status.
- Display all plans and search by name or duration.
- Update plan name, duration, fee, and active status.
- Deactivate or delete a plan only if existing membership history remains intact.

C. Membership operations

- Assign a plan to an existing member and create a unique MembershipId.
- Calculate EndDate from StartDate and plan duration.
- Display all memberships and retrieve one by ID.
- Renew a membership and calculate the new period: extend from current expiry if still active, otherwise start from the renewal date.
- Cancel a membership and update its status.
- Find a member's current membership and show whether it is Active, Expired, or Cancelled.
- Do not allow a new membership to use an inactive plan.

D. Payment management

- Record a payment for a membership with unique PaymentId.
- Display and search payments by ID, member, membership, date, and status.
- Record payment amount, date, method, and status.
- Update payment status where permitted and reject negative or zero payment amounts.
- Require successful payment before activating a membership.

E. Data persistence

- Save members, plans, memberships, and payments to JSON.
- Load saved records and maintain relationships through IDs.
- Handle missing files and malformed JSON safely.

Business rules and validation

- Member, Plan, Membership, and Payment IDs must be unique.
- Member age must be within the configured valid range; phone number is required.

- Plan duration and fee must be greater than zero.
- Membership end date must not precede start date.
- Renewal must reference an existing member and valid plan.
- Inactive plans cannot be used for new memberships.
- Payment amount must be greater than zero; pending or failed payments do not count as successful.
- Historical membership and payment records must remain understandable if a plan fee changes.

Required reports using LINQ

- Find active, expired, and cancelled memberships.
- Find memberships expiring within the next 7 or 30 days.
- Group memberships by plan and count each group.
- Calculate successful payment revenue for a selected date range.
- Find the most popular plan by membership count.
- Project a member report showing member name, plan name, start date, end date, and status.

USE CASE 4 — Event Registration Management System

Assigned students: Students 31–40

Question

An event organizer needs a console application to manage events, participants, registrations, capacity, and fees. The system must prevent duplicate registration and overbooking while producing event-level reports.

Required data entities

Event: EventId, Title, Category, Venue, EventDate, RegistrationFee, Capacity, Status

Participant: ParticipantId, FullName, Email, PhoneNumber

Registration: RegistrationId, EventId, ParticipantId, RegistrationDate, FeeCharged, Status

Required operations

A. Event management

- Create an event with unique ID, title, category, venue, date, fee, capacity, and status.
- Display all events and retrieve an event by ID.
- Search events by title, category, or date.
- Update event details, fee, venue, date, and capacity subject to validation.
- Cancel an event and update affected registrations.
- Delete an event only when registration records will not be orphaned.

B. Participant management

- Register a participant with unique ParticipantId and required contact details.
- Display all participants and retrieve a participant by ID.
- Search by name or email.
- Update participant details.
- Delete a participant only when registration history remains consistent.

C. Registration management

- Register an existing participant for an open event.
- Generate a unique RegistrationId.
- Prevent duplicate active registrations by the same participant for the same event.
- Check remaining capacity before confirming registration.
- View registrations by ID, event, participant, date, and status.
- Cancel a registration and release its seat.
- Calculate and preserve the fee charged at registration.
- Reject new registrations with a clear message when capacity is full.

D. Data persistence

- Save events, participants, and registrations to JSON.
- Load data at startup and retain event/participant references through IDs.
- Handle missing files and malformed JSON without erasing data.

Business rules and validation

- Event, Participant, and Registration IDs must be unique.
- Event date must be valid; registration fee must be non-negative; capacity must be greater than zero.
- Only Open events with future dates may accept new registrations.
- Active registrations must never exceed event capacity.
- Cancelled registrations do not consume a seat.
- Cancelled events cannot accept registrations.
- An event's capacity cannot be reduced below its current number of active registrations.

- Fee changes must not change the fee charged on existing registrations.

Required reports using LINQ

- Find upcoming events and sort them by date.
- Find events with no remaining seats or fewer than a chosen number of seats remaining.
- Group active registrations by event and count participants.
- Calculate registration revenue by event using recorded FeeCharged values.
- Find the event with the highest active registration count.
- Project a registration list with participant name, event title, registration date, and status.

USE CASE 5 — Courier and Shipment Tracking System

Assigned students: Students 41–50

Question

A courier company needs to manage customer records, parcel bookings, shipment tracking, charges, and delivery status. The application must enforce valid status transitions and provide operational shipment reports.

Required data entities

Customer: CustomerId, FullName, PhoneNumber, Email, Address

Shipment: ShipmentId, TrackingNumber, SenderCustomerId, RecipientName, RecipientPhone, Destination, WeightKg, DeliveryType, BookedAt, Status, ShippingCharge, DeliveredAt

ShipmentStatusHistory: HistoryId, ShipmentId, Status, UpdatedAt, Note (optional)

Required operations

A. Customer management

- Register a customer with unique CustomerId and required contact details.
- Display all customers and retrieve a customer by ID.
- Search customers by name, phone, or email.
- Update customer contact details and address.
- Delete a customer only when doing so will not break shipment history; otherwise reject deletion.

B. Shipment booking and retrieval

- Create a shipment with a unique ShipmentId and unique TrackingNumber.
- Capture sender customer, recipient details, destination, parcel weight, delivery type, and booking date.
- Display all shipments and retrieve a shipment by ID or tracking number.
- Search shipments by sender, recipient, destination, booking date, and status.
- Update shipment details before dispatch.
- Cancel a shipment only when its status allows cancellation.
- Preserve the shipping charge calculated at booking unless an allowed pre-dispatch edit requires recalculation.

C. Shipping charges and delivery status

- Calculate shipping charges using weight, delivery type, and destination category according to a documented rate table.
- Support at least two delivery types, such as Standard and Express, with different rates.
- Update shipment status through Booked, Dispatched, In Transit, Delivered, or Cancelled.
- Enforce valid status transitions; a Delivered shipment cannot return to In Transit.
- Record a delivery timestamp when status becomes Delivered.
- Maintain a status history record for every successful status change.
- Reject invalid weights, unknown customers, invalid delivery types, and unsupported status transitions.

D. Data persistence

- Save customers, shipments, and status history to JSON.
- Load saved data at startup and retain relationships through IDs.
- Handle missing files and malformed JSON safely.

Business rules and validation

- Customer, Shipment, Tracking, and History IDs must be unique where applicable.
- Weight must be greater than zero; shipping rates and calculated charges must be non-negative.
- Only existing sender/customer records can be referenced.
- Delivery type and status must be chosen from supported values.
- Status transitions must follow a documented transition rule.

- DeliveredAt must be set when delivered and must not be set for shipments that have not been delivered.
- Cancelled shipments cannot continue through delivery states.
- Shipment history must match successful status changes and must not be silently rewritten.

Required reports using LINQ

- Find shipments by status and sort by booking date.
- Find shipments going to a selected destination.
- Group shipments by delivery type and count each type.
- Calculate total charges for shipments booked in a selected date range.
- Find the heaviest shipment and the shipment with the highest charge.
- Count delivered, in-transit, pending, and cancelled shipments.
- Project a tracking summary with tracking number, recipient, destination, current status, charge, and delivery date.